



Uniwersytet Rzeszowski  
Wydział Nauk Ścisłych i Technicznych

## **PROGRAMOWANIE OBIEKTOWE 2**

*System zarządzania ośrodkiem szkolenia kierowców*

Damian Kania

Nr albumu: 134917

Kierunek studiów: Informatyka

Prowadzący: mgr inż. Ewa Żesławska

Rzeszów, 2026





## Streszczenie

Celem tego projektu było stworzenie aplikacji desktopowej, która ma ułatwić pracę ośrodka szkolenia kierowców. Aplikacja powstała w języku C# z wykorzystaniem technologii WPF i wzorca MVVM, a dane przechowywane są w lokalnej bazie SQL Server obsługiwanej przez Entity Framework Core.

W aplikacji zostały zaimplementowane trzy role użytkowników. Administrator może dodawać, edytować i usuwać kursantów, instruktorów oraz pojazdy, a także zarządzać wszystkimi rezerwacjami. Instruktor widzi swój grafik na dany tydzień i może anulować własne jazdy. Kursant może sam rezerwować dostępne terminy jazd przez interaktywny kalendarz.

Projekt zawiera mechanizmy walidacji danych, bezpieczne przechowywanie haseł oraz ochronę integralności danych w bazie.

# Abstract

The goal of this project was to create a desktop application to help manage a driving school (OSK). The app was built in C# using WPF and the MVVM pattern, with data stored in a local SQL Server database accessed through Entity Framework Core.

The application has three user roles. The administrator can add, edit and delete students, instructors and vehicles, and can manage all driving reservations. The instructor can view their weekly schedule and cancel their own driving sessions. The student can independently book available driving slots using an interactive calendar.

The project includes data validation mechanisms, secure password storage and data integrity protection in the database.

## Spis treści

|                                                      |           |
|------------------------------------------------------|-----------|
| <b>Streszczenie.....</b>                             | <b>4</b>  |
| <b>Abstract.....</b>                                 | <b>5</b>  |
| <b>1. Wprowadzenie i analiza problemu .....</b>      | <b>8</b>  |
| 1.1. Cel i zakres projektu .....                     | 8         |
| 1.2. Przegląd istniejących rozwiązań.....            | 8         |
| <b>2. Wymagania systemowe .....</b>                  | <b>9</b>  |
| 2.1. Wymagania funkcjonalne .....                    | 9         |
| 2.1.1. Administrator .....                           | 9         |
| 2.1.2. Instruktor.....                               | 9         |
| 2.1.3. Kursant .....                                 | 9         |
| 2.2. Założenia dotyczące działania systemu.....      | 9         |
| 2.3. Wymagania niefunkcjonalne .....                 | 10        |
| <b>3. Architektura i struktura aplikacji.....</b>    | <b>11</b> |
| 3.1. Ogólna struktura warstwowa .....                | 11        |
| 3.2. Diagram pakietów .....                          | 11        |
| 3.3. Przepływ sterowania.....                        | 11        |
| 3.4. Wzorzec MVVM w projekcie .....                  | 11        |
| 3.5. Diagram klas .....                              | 12        |
| 3.6. Mechanizmy walidacji danych .....               | 12        |
| 3.7. Obsługa błędów i wyjątków .....                 | 12        |
| <b>4. Technologie wykorzystane w projekcie .....</b> | <b>13</b> |
| 4.1. Technologie interfejsu użytkownika.....         | 13        |
| 4.2. Technologie dostępu do danych .....             | 13        |
| 4.3. Dodatkowe biblioteki .....                      | 13        |
| 4.4. Narzędzia deweloperskie .....                   | 13        |
| <b>5. Projekt bazy danych .....</b>                  | <b>14</b> |
| 5.1. Diagram ERD .....                               | 14        |
| 5.2. Normalizacja.....                               | 14        |
| 5.3. Relacje i integralność danych .....             | 14        |
| <b>6. Implementacja aplikacji.....</b>               | <b>15</b> |
| 6.1. System uwierzytelniania.....                    | 15        |
| 6.2. Walidacja numeru PESEL .....                    | 15        |
| 6.3. Ochrona przed wyścigiem danych (TOCTOU).....    | 15        |
| 6.4. Kalendarz rezerwacji .....                      | 16        |
| 6.5. Unikalność loginów przy edycji.....             | 16        |
| 6.6. Realizacja operacji CRUD.....                   | 16        |

|                                                       |           |
|-------------------------------------------------------|-----------|
| 6.7. Organizacja nawigacji .....                      | 16        |
| 6.8. Interfejs użytkownika i stylizacja .....         | 17        |
| <b>7. Prezentacja warstwy użytkowej projektu.....</b> | <b>18</b> |
| 7.1. Ekran logowania .....                            | 18        |
| 7.2. Główne okno aplikacji.....                       | 18        |
| 7.3. Moduł zarządzania kursantami.....                | 19        |
| 7.4. Moduł zarządzania instruktorami .....            | 20        |
| 7.5. Moduł zarządzania pojazdami .....                | 21        |
| 7.6. Moduł kalendarza rezerwacji.....                 | 22        |
| 7.7. Przykładowy scenariusz działania .....           | 24        |
| 7.8. Scenariusze testowania .....                     | 24        |
| <b>8. Harmonogram realizacji projektu .....</b>       | <b>25</b> |
| <b>9. Instrukcja uruchomienia projektu .....</b>      | <b>26</b> |
| 9.1. Wymagania systemowe .....                        | 26        |
| 9.2. Uruchomienie aplikacji .....                     | 26        |
| 9.3. Dane logowania .....                             | 26        |
| <b>10. Podsumowanie i wnioski.....</b>                | <b>27</b> |
| 10.1. Ocena realizacji celu.....                      | 27        |
| 10.2. Wnioski.....                                    | 27        |
| 10.3. Napotkane problemy .....                        | 27        |
| 10.4. Możliwości rozbudowy .....                      | 27        |
| 10.5. Podsumowanie .....                              | 27        |
| <b>Bibliografia .....</b>                             | <b>28</b> |
| <b>Spis rysunków .....</b>                            | <b>29</b> |
| <b>Spis tabel.....</b>                                | <b>30</b> |
| <b>Spis listingów .....</b>                           | <b>31</b> |

# 1. Wprowadzenie i analiza problemu

## 1.1. Cel i zakres projektu

Celem projektu było stworzenie aplikacji desktopowej wspomagającej pracę ośrodka szkolenia kierowców. Tradycyjne OSK prowadzą dokumentację w formie papierowej lub w arkuszach kalkulacyjnych, co jest czasochłonne i podatne na błędy. Proponowana aplikacja automatyzuje najważniejsze czynności – zarządzanie kursantami, instruktorami, pojazdami i rezerwacjami jazd.

Zakres projektu obejmuje:

- aplikację okienkową dla systemu Windows,
- relacyjną bazę danych obsługiwaną przez kod C#,
- system logowania z trzema rolami użytkowników,
- kalendarz rezerwacji terminów jazd,
- walidację wprowadzanych danych (m.in. numer PESEL),
- bezpieczne przechowywanie haseł i ochronę danych.

## 1.2. Przegląd istniejących rozwiązań

Na rynku dostępnych jest kilka aplikacji przeznaczonych dla ośrodków szkolenia kierowców. Poniżej omówiono trzy przykłady.

**iMSZ (Internetowy Moduł Szkoły Jazdy)** to popularne w Polsce rozwiązanie webowe oferujące zarządzanie kursantami i harmonogramem jazd. Działa w przeglądarce, lecz wymaga stałego dostępu do internetu i opłaty miesięcznej.

**OSK Zintegrowany** to aplikacja desktopowa dla Windows z modulem kalendarza. Jest to oprogramowanie płatne i nie wspiera nowoczesnych standardów bezpieczeństwa.

**Proton OSK** to również aplikacja okienkowa z modulem rezerwacji i raportami. Wymaga oddzielnej licencji i nie oferuje bezpłatnego wdrożenia.

Projekt realizowany w ramach niniejszej pracy stanowi uproszczoną, ale w pełni działającą alternatywę bez zewnętrznych kosztów.

## **2. Wymagania systemowe**

### **2.1. Wymagania funkcjonalne**

Wymagania funkcjonalne określają konkretne zachowania systemu z perspektywy każdej z trójki ról użytkowników.

#### **2.1.1. Administrator**

- Logowanie do systemu przy użyciu loginu i hasła.
- Dodawanie, edytowanie i usuwanie kursantów (imię, nazwisko, PESEL, login, hasło, kategoria prawa jazdy).
- Dodawanie, edytowanie i usuwanie instruktorów (imię, nazwisko, numer legitymacji, telefon, login, hasło, uprawnienia do kategorii).
- Dodawanie, edytowanie i usuwanie pojazdów (marka, model, numer rejestracyjny, kategoria).
- Przeglądanie pełnego kalendarza rezerwacji z możliwością filtrowania po kursancie, instruktorze i pojeździe.
- Rezerwowanie i anulowanie jazd w imieniu dowolnego kursanta.

#### **2.1.2. Instruktor**

- Logowanie do systemu.
- Przeglądanie własnego grafiku na bieżący tydzień i tygodnie sąsiednie.
- Anulowanie własnych zarezerwowanych jazd.

#### **2.1.3. Kursant**

- Logowanie do systemu.
- Przeglądanie kalendarza dostępnych terminów jazdy.
- Rezerwowanie wolnego terminu jazdy.
- Anulowanie własnej zarezerwowanej jazdy.

### **2.2. Założenia dotyczące działania systemu**

- Aplikacja przeznaczona jest do użytku lokalnego na jednym komputerze z systemem Windows.
- Każdy użytkownik musi się zalogować – dostęp bez logowania nie jest możliwy.
- Dane przechowywane są w lokalnej bazie danych – nie jest wymagane połączenie z internetem.
- Aplikacja obsługuje jednocześnie jedną aktywną sesję użytkownika.
- Baza danych tworzona jest automatycznie przy pierwszym uruchomieniu.

## 2.3. Wymagania niefunkcjonalne

**Tabela 2.1.** Wymagania niefunkcjonalne systemu

| Kategoria      | Opis                                                                                                                                                                                                           |
|----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Bezpieczeństwo | Hasła użytkowników są przechowywane w zaszyfrowanej formie. Loginy muszą być unikalne w obrębie każdej roli (kursanci, instruktorzy, administratorzy osobno). Numery PESEL muszą być unikalne wśród kursantów. |
| Niezawodność   | Błędy są przechwytywane i komunikowane użytkownikowi w formie czytelnych komunikatów.                                                                                                                          |
| Wydajność      | Czas odpowiedzi interfejsu poniżej 1 sekundy dla typowych operacji.                                                                                                                                            |
| Przenoszalność | Aplikacja działa na systemie Windows 10 i Windows 11.                                                                                                                                                          |
| Utrzymywalność | Kod aplikacji jest podzielony na warstwy, co ułatwia rozbudowę o nowe funkcje.                                                                                                                                 |

## 3. Architektura i struktura aplikacji

### 3.1. Ogólna struktura warstwowa

Aplikacja podzielona jest na trzy logiczne warstwy zgodnie ze wzorcem Model–View–ViewModel (MVVM):

- **Warstwa modelu** – klasy encji Entity Framework mapowane na tabele bazy danych (Models/): Kursant, Instruktor, Pojazd, Administrator, RezerwacjaJazdy.
- **Warstwa widoku** – pliki XAML i ich *code-behind* (Views/): okna i kontrolki użytkownika.
- **Warstwa ViewModel** – klasy dziedziczące po ViewModelBase (ViewModels/): logika biznesowa, walidacja, komunikacja z bazą.

Komunikacja pomiędzy widokiem a ViewModel odbywa się przez mechanizm **wiązań danych** (data binding) WPF oraz wzorzec poleceń ICommand zrealizowany klasą RelayCommand.

### 3.2. Diagram pakietów

Tabela 3.1. Struktura katalogów projektu

| Katalog     | Zawartość                                       |
|-------------|-------------------------------------------------|
| Models/     | Klasy encji (5 plików)                          |
| Data/       | OskDbContext – kontekst EF Core                 |
| ViewModels/ | 5 klas ViewModel + ViewModelBase + RelayCommand |
| Views/      | Okna XAML i code-behind (7 plików)              |
| Migrations/ | Migracje EF Core (historia schematu bazy)       |

### 3.3. Przepływ sterowania

Po uruchomieniu aplikacji wyświetlane jest okno logowania. Po pomyślnym uwierzytelnieniu otwierane jest główne okno aplikacji, które w zależności od roli użytkownika wyświetla odpowiednie opcje menu i wczytuje odpowiedni widok do centralnego obszaru.

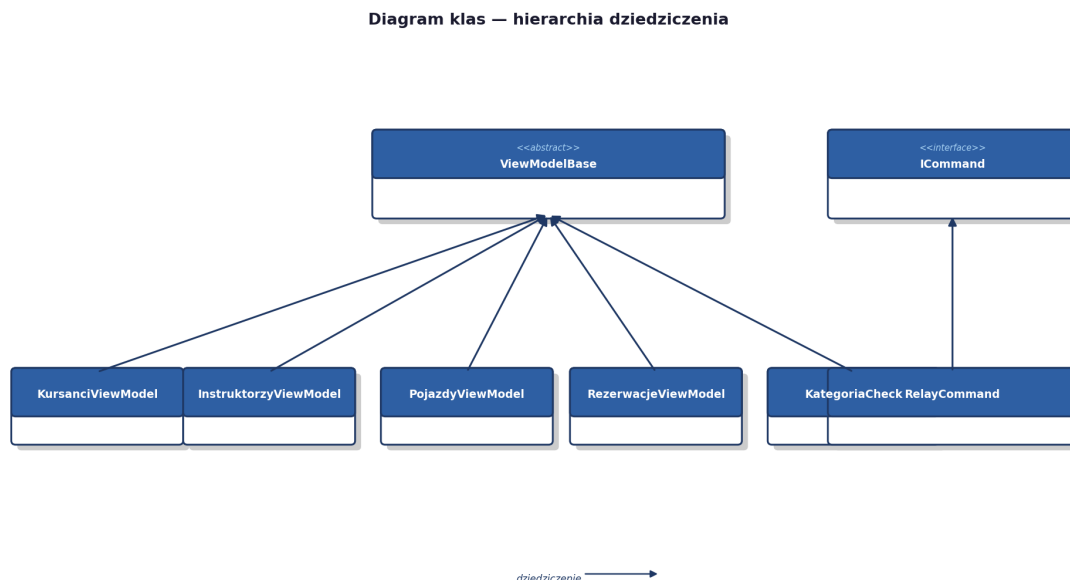
Kliknięcie przycisku menu podменя zawartość obszaru roboczego na jeden z widoków: kursanci, instruktorzy, pojazdy lub rezerwacje. Każdy widok ładuje niezbędne dane z bazy przy swojej inicjalizacji.

### 3.4. Wzorzec MVVM w projekcie

Zastosowanie wzorca MVVM pozwala na wyraźne rozdzielenie interfejsu graficznego od logiki aplikacji. Dzięki temu zmiany w wyglądzie okna nie wpływają na logikę działania programu i odwrotnie. Przyciski w oknach są bezpośrednio powiązane z odpowiednimi akcjami w kodzie, bez dodatkowego kodu pośredniczącego.

### 3.5. Diagram klas

Poniższy diagram przedstawia hierarchię dziedziczenia w projekcie. Klasa `ViewModelBase` jest klasą bazową, po której dziedziczą wszystkie `ViewModele`. Klasa `RelayCommand` realizuje interfejs `ICommand`.



Rysunek 3.1. Diagram klas

### 3.6. Mechanizmy walidacji danych

Walidacja danych wejściowych odbywa się w warstwie `ViewModel`, przed wykonaniem jakiegokolwiek operacji na bazie danych. Sprawdzane są:

- poprawność numeru PESEL (długość, cyfry, suma kontrolna),
- unikalność loginu i numeru PESEL w bazie danych,
- wypełnienie wszystkich wymaganych pól formularza,
- wartość daty rezerwacji – termin nie może być z przeszłości.

W przypadku błędu użytkownik otrzymuje czytelny komunikat opisujący przyczynę problemu, a operacja nie jest wykonywana.

### 3.7. Obsługa błędów i wyjątków

Operacje bazodanowe są opakowane w bloki `try-catch`, dzięki czemu wyjątki (np. błąd połączenia z bazą lub naruszenie klucza obcego) są przechwytywane i prezentowane użytkownikowi w postaci okna dialogowego z opisem problemu, zamiast powodować awarię aplikacji.

## 4. Technologie wykorzystane w projekcie

### 4.1. Technologie interfejsu użytkownika

Interfejs graficzny został zrealizowany przy użyciu technologii **Windows Presentation Foundation (WPF)**. Układy wizualne zdefiniowano w języku **XAML**, co pozwala na łatwą modyfikację wyglądu bez ingerencji w logikę aplikacji. WPF został wybrany ze względu na dojrzałość platformy, bogaty zestaw kontrolki i natywne wsparcie dla wzorca MVVM.

### 4.2. Technologie dostępu do danych

Dostęp do bazy danych zapewnia **Entity Framework Core** w podejściu *Code First*. Struktura bazy danych definiowana jest bezpośrednio w kodzie C# jako klasy modeli, a narzędzie automatycznie generuje i stosuje migracje schematu. Wybór EF Core pozwolił uniknąć pisania surowych zapytań SQL i przyspieszył implementację operacji na danych.

Jako silnik bazy danych zastosowano **SQL Server LocalDB**, który nie wymaga instalacji pełnego serwera. Baza działa lokalnie na komputerze użytkownika, co upraszcza wdrożenie aplikacji.

### 4.3. Dodatkowe biblioteki

Do szyfrowania haseł użyto biblioteki **BCrypt.Net-Next**. Hasła nigdy nie są przechowywane w formie jawnej – zapisywany jest wyłącznie wynik funkcji skrótu. BCrypt.Net został wybrany jako powszechnie stosowany standard zabezpieczania haseł w aplikacjach.

### 4.4. Narzędzia deweloperskie

**Tabela 4.1.** Narzędzia użyte w projekcie

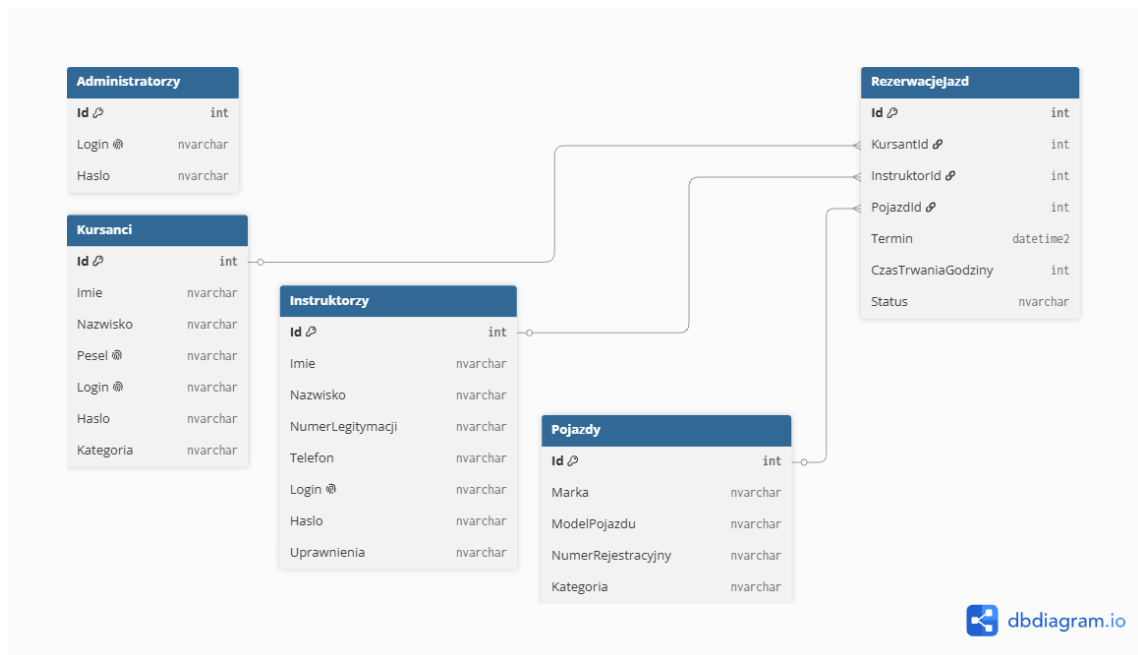
| Narzędzie          | Zastosowanie                             |
|--------------------|------------------------------------------|
| Visual Studio 2022 | Główne środowisko programistyczne (IDE)  |
| .NET 10 SDK        | Platforma uruchomieniowa i kompilator C# |
| Git                | Kontrola wersji kodu źródłowego          |
| NuGet              | Zarządzanie bibliotekami zewnętrznymi    |
| SQL Server LocalDB | Lokalna baza danych                      |

## 5. Projekt bazy danych

Baza danych systemu składa się z pięciu tabel: Kursanci, Instruktorzy, Pojazdy, RezerwacjeJazd i Administratorzy. Tabela RezerwacjeJazd jest tabelą centralną – łączy ze sobą pozostałe trzy przez powiązania.

### 5.1. Diagram ERD

Poniższy diagram przedstawia strukturę bazy danych oraz powiązania między tabelami.



Rysunek 5.1. Diagram ERD bazy danych systemu OSK

### 5.2. Normalizacja

Schemat bazy danych spełnia trzecią postać normalną (3NF). Każda tabela przechowuje dane dotyczące tylko jednego obiektu: osobno kursanci, osobno instruktorzy, osobno pojazdy. Powiązania między nimi realizowane są przez klucze obce w tabeli rezerwacji, bez powtórzenia danych.

### 5.3. Relacje i integralność danych

Każda rezerwacja musi być powiązana z istniejącym kursantem, instruktorem i pojazdem. System blokuje usunięcie kursanta, instruktora lub pojazdu, jeżeli ma on przypisane jazdy w systemie – zapobiega to przypadkowej utracie danych.

Loginy użytkowników oraz numer PESEL kursanta muszą być unikalne. Wymuszane jest to na poziomie bazy danych, co uniemożliwia duplikaty nawet przy jednoczesnym dostępie.

Struktura bazy została utworzona przy użyciu mechanizmu migracji – zmiany schematu są wersjonowane i można je w każdej chwili odtworzyć.

## 6. Implementacja aplikacji

### 6.1. System uwierzytelniania

Okno logowania (`LoginWindow`) pobiera login, hasło i wybraną rolę z formularza. Następnie wyszukuje rekord w odpowiedniej tabeli bazy danych i weryfikuje hasło metodą `BCrypt.Verify`. Fragment kodu przedstawia Listing 6.1.

**Listing 6.1.** Weryfikacja hasła bcrypt podczas logowania

```
1 var kursant = context.Kursanci
2   .FirstOrDefault(k => k.Login == login);
3 if (kursant != null
4     && BCrypt.Net.BCrypt.Verify(haslo, kursant.Haslo))
5 {
6     OtworzMainWindow("Kursant", login);
7     return;
8 }
```

### 6.2. Walidacja numeru PESEL

Przy dodawaniu lub edycji kursanta sprawdzana jest poprawność numeru PESEL. Algorytm weryfikuje 11-cyfrową długość, cyfrowść wszystkich znaków oraz sumę kontrolną opartą na wagach  $\{1, 3, 7, 9, 1, 3, 7, 9, 1, 3\}$ .

**Listing 6.2.** Algorytm weryfikacji sumy kontrolnej PESEL

```
1 private static bool WalidujSumeKontrolnaPesel(string pesel)
2 {
3     int[] wagi = { 1, 3, 7, 9, 1, 3, 7, 9, 1, 3 };
4     int suma = 0;
5     for (int i = 0; i < 10; i++)
6         suma += (pesel[i] - '0') * wagi[i];
7     int cyfraKontrolna = (10 - (suma % 10)) % 10;
8     return cyfraKontrolna == (pesel[10] - '0');
9 }
```

### 6.3. Ochrona przed wyścigiem danych (TOCTOU)

Operacja usunięcia rekordu realizowana jest w dwóch etapach: najpierw wyświetlane jest pytanie potwierdzające, a następnie – po potwierdzeniu przez użytkownika – otwierany jest nowy kontekst bazy, w którym wykonywane są oba kroki: pobranie rekordu i jego usunięcie. Ponieważ aplikacja przeznaczona jest do użytku lokalnego z jedną aktywną sesją (por. rozdział 2), ryzyko równoczesnej modyfikacji przez innego użytkownika nie występuje. Zastosowane podejście zabezpiecza przed sytuacją, w której rekord mógłby zostać już usunięty w tle między wyświetleniem pytania a faktycznym wykonaniem operacji.

**Listing 6.3.** Zabezpieczenie przed wyścigiem danych przy usuwaniu

```
1 var wynik = MessageBox.Show("Czy na pewno usunac kursanta?",
2     "Potwierdzenie", MessageBoxButton.YesNo, MessageBoxImage.Question);
```

```
3 if (wynik != MessageBoxResult.Yes) return;
4
5 using var context = new OskDbContext();
6 var doUsuniecia = context.Kursanci.Find(kursant.Id);
7 if (doUsuniecia == null)
8 {
9     MessageBox.Show("Kursant nie istnieje w bazie.",
10         "Bład", MessageBoxButton.OK, MessageBoxImage.Warning);
11     return;
12 }
13 context.Kursanci.Remove(doUsuniecia);
14 context.SaveChanges();
```

## 6.4. Kalendarz rezerwacji

Kalendarz tygodniowy wyświetla terminy jazd w zakresie godzinowym 8–18 dla każdego dnia roboczego. Każdy termin ma przypisany kolor określający jego status – zajęty lub wolny. Administrator może filtrować widok kalendarza po kursancie, instruktora lub pojeździe. Każda zmiana filtra odświeża wyświetlane terminy.

## 6.5. Unikalność loginów przy edycji

Podczas edycji konta sprawdzane jest, czy nowy login nie jest już używany przez inną osobę. Własny login jest przy tym pomijany, dzięki czemu edycja bez zmiany loginu nie powoduje błędu:

**Listing 6.4.** Sprawdzanie unikalności loginu przy edycji

```
1 bool loginZajety = context.Kursanci.Any(
2     k2 => k2.Login == NowyLogin && k2.Id != edytowany.Id);
3 if (loginZajety)
4 {
5     MessageBox.Show("Podany login jest już zajęty.",
6         "Bład walidacji", MessageBoxButton.OK, MessageBoxImage.Warning);
7     return;
8 }
```

## 6.6. Realizacja operacji CRUD

Każdy moduł zarządzania (kursanci, instruktorzy, pojazdy) realizuje pełny zestaw operacji CRUD:

- **Create** – formularz w oknie dialogowym, zapis do bazy po walidacji danych.
- **Read** – ładowanie i wyświetlanie danych przy otwarciu widoku.
- **Update** – otwarcie okna edycji z wypełnionymi danymi, zapis zmienionego rekordu.
- **Delete** – potwierdzenie operacji przez użytkownika, usunięcie rekordu z bazy z zachowaniem reguł integralności.

## 6.7. Organizacja nawigacji

Po zalogowaniu otwierane jest główne okno (MainWindow) z bocznym menu. Każdy przycisk menu podmienia zawartość centralnego obszaru na odpowiedni widok. Widoczność przycisków jest kontrolowana

na podstawie roli – kursant i instruktor widzą tylko własne opcje, administrator ma dostęp do wszystkich modułów. Okna dialogowe (dodawanie, edycja, potwierdzenie usunięcia) otwierane są jako modalne – blokują główne okno do czasu zamknięcia.

## 6.8. Interfejs użytkownika i stylizacja

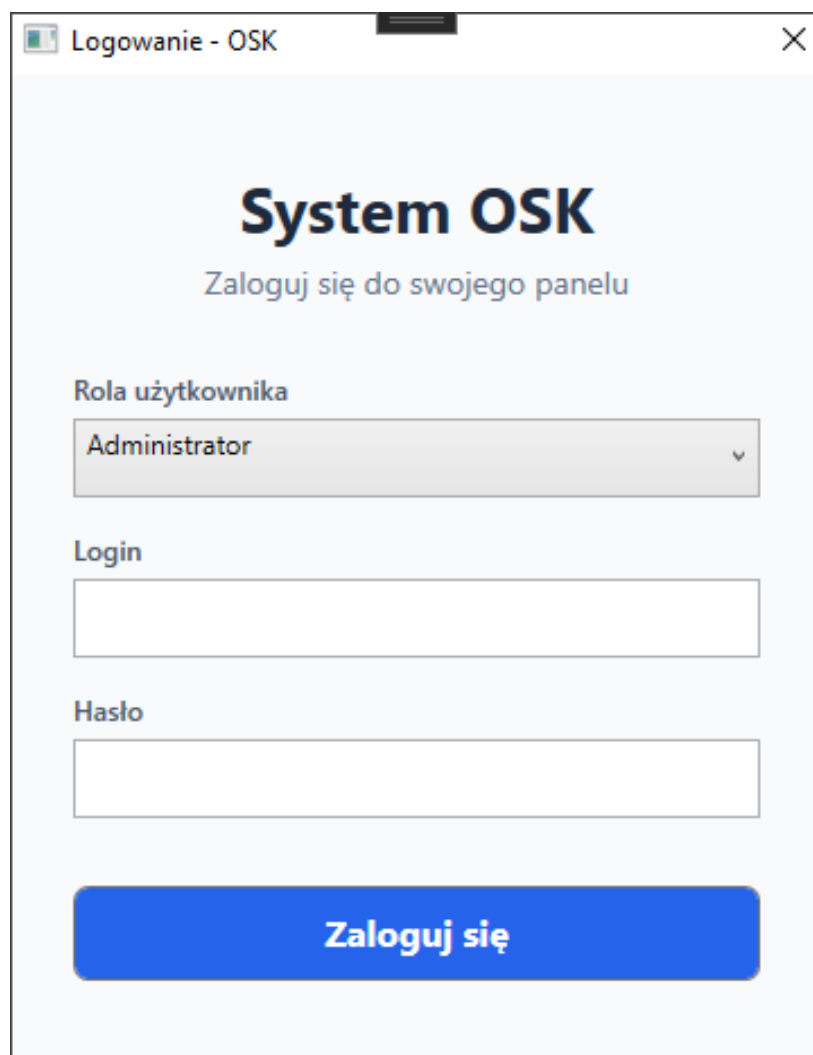
Interfejs zaprojektowany został z myślą o czytelności i prostocie obsługi. Zastosowano neutralną kolorystykę – białe tło, ciemnoszare nagłówki i kolorowe wyróżnienia statusów w kalendarzu (zielony/czerwony/szary). Układy okien zdefiniowano w XAML z użyciem `Grid` i `StackPanel`, co zapewnia prawidłowe skalowanie przy różnych rozdzielczościach ekranu.

## 7. Prezentacja warstwy użytkowej projektu

Rozdział przedstawia wygląd i działanie kolejnych ekranów aplikacji wraz z opisem najważniejszych funkcjonalności dostępnych dla każdej z trójki ról użytkowników.

### 7.1. Ekran logowania

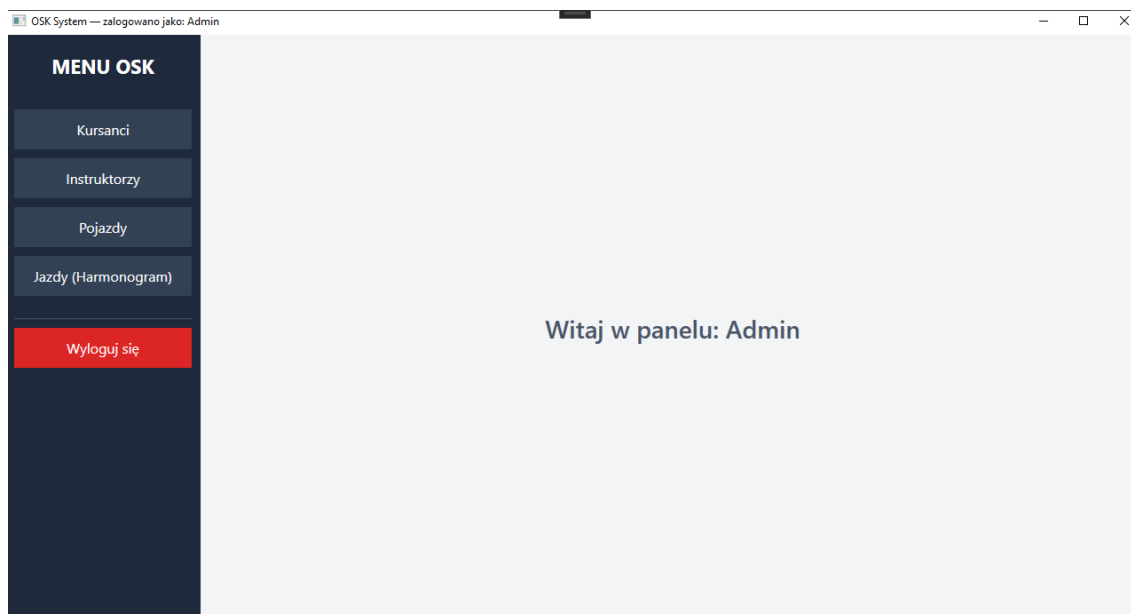
Po uruchomieniu aplikacji wyświetlane jest okno logowania (rys. 7.1). Użytkownik wybiera swoją rolę z listy rozwijanej (*Administrator*, *Instruktor*, *Kursant*), wpisuje login oraz hasło i klika przycisk **Zaloguj**. W przypadku błędnych danych wyświetlany jest komunikat „Niepoprawny login lub hasło!”.



Rysunek 7.1. Ekran logowania

### 7.2. Główne okno aplikacji

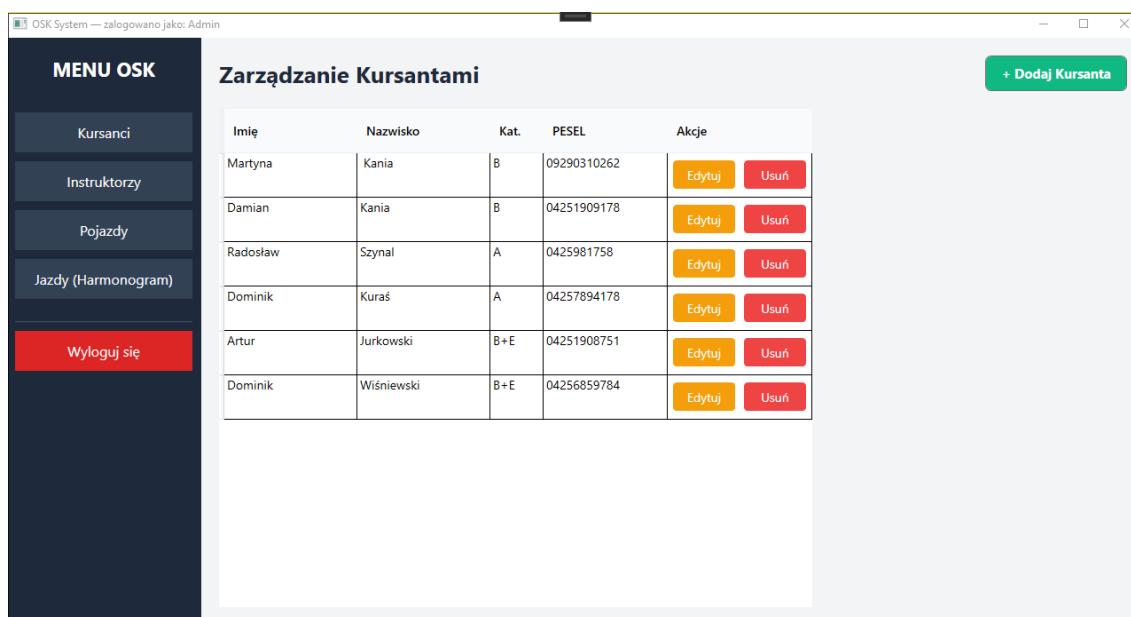
Po zalogowaniu się jako administrator widoczne są wszystkie cztery przyciski menu bocznego (rys. 7.2). Kursant i instruktor widzą wyłącznie przycisk własnych rezerwacji lub grafiku – pozostałe opcje są ukryte.



Rysunek 7.2. Główne okno aplikacji – widok administratora

### 7.3. Moduł zarządzania kursantami

Lista kursantów wyświetlana jest w formie tabeli (rys. 7.3). Dla każdego wiersza dostępne są przyciski **Edytuj** i **Usuń**. Kliknięcie **Dodaj kursanta** otwiera modalne okno dialogowe (rys. 7.4) z formularzem: imię, nazwisko, PESEL, login, hasło oraz kategoria prawa jazdy. Przed zapisem sprawdzana jest poprawność numeru PESEL (algorytm sumy kontrolnej) oraz unikalność loginu i numeru PESEL w bazie. Próba wpisania błędnego numeru PESEL skutkuje komunikatem przedstawionym na rysunku 7.5.



Rysunek 7.3. Lista kursantów

OSK System — zalogowano jako: Admin

**MENU OSK**

- Kursanci
- Instruktorzy
- Pojazdy
- Jazdy (Harmonogram)
- Wyloguj się

**Zarządzanie Kursantami**

| Imię     | Nazwisko   |      |
|----------|------------|------|
| Martyna  | Kania      | Usuń |
| Damian   | Kania      | Usuń |
| Radosław | Szynał     | Usuń |
| Dominik  | Kuraś      | Usuń |
| Artur    | Jurkowski  | Usuń |
| Dominik  | Wiśniewski | Usuń |

**Formularz Kursanta**

Imię\*

Nazwisko\*

Kategoria Prawa Jazdy

PESEL\* (11 cyfr)

Login

Hasło (pozostaw puste aby nie zmieniać)

Zapisz

+ Dodaj Kursanta

Rysunek 7.4. Formularz dodawania kursanta

OSK System — zalogowano jako: Admin

**MENU OSK**

- Kursanci
- Instruktorzy
- Pojazdy
- Jazdy (Harmonogram)
- Wyloguj się

**Zarządzanie Kursantami**

| Imię     | Nazwisko   |      |
|----------|------------|------|
| Martyna  | Kania      | Usuń |
| Damian   | Kania      | Usuń |
| Radosław | Szynał     | Usuń |
| Dominik  | Kuraś      | Usuń |
| Artur    | Jurkowski  | Usuń |
| Dominik  | Wiśniewski | Usuń |
| Jan      | Kowalski   | Usuń |

**Formularz Kursanta**

Imię\*

Nazwisko\*

Kategoria Prawa Jazdy

PESEL\*

Login

Hasło (pozostaw puste aby nie zmieniać)

Zapisz

+ Dodaj Kursanta

**Błądny PESEL**

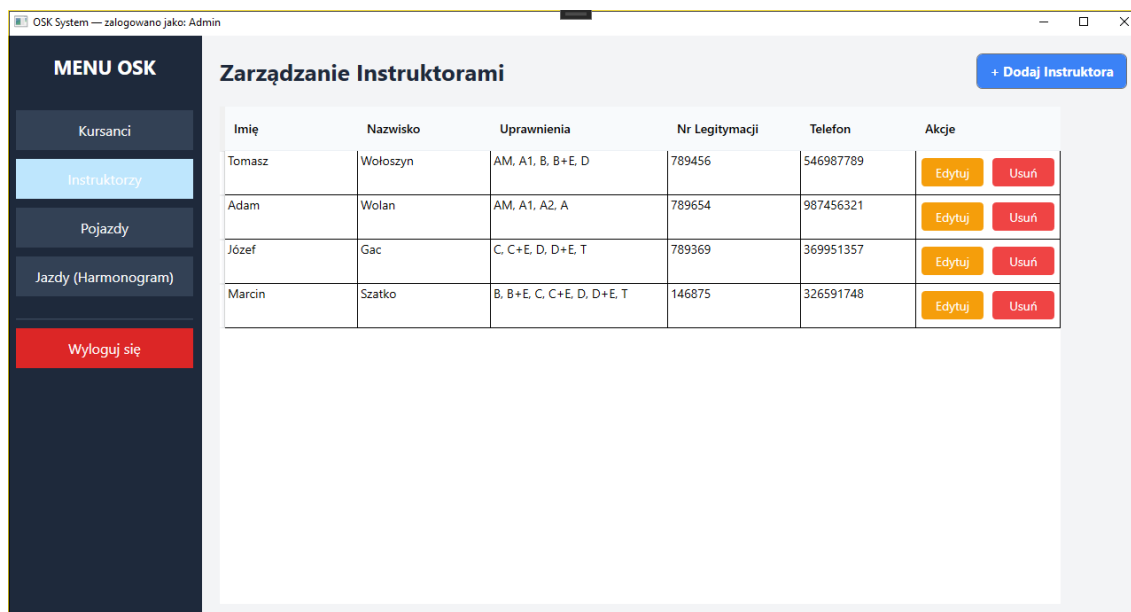
Podany PESEL jest nieprawidłowy — sprawdź czy wpisałeś go poprawnie.

OK

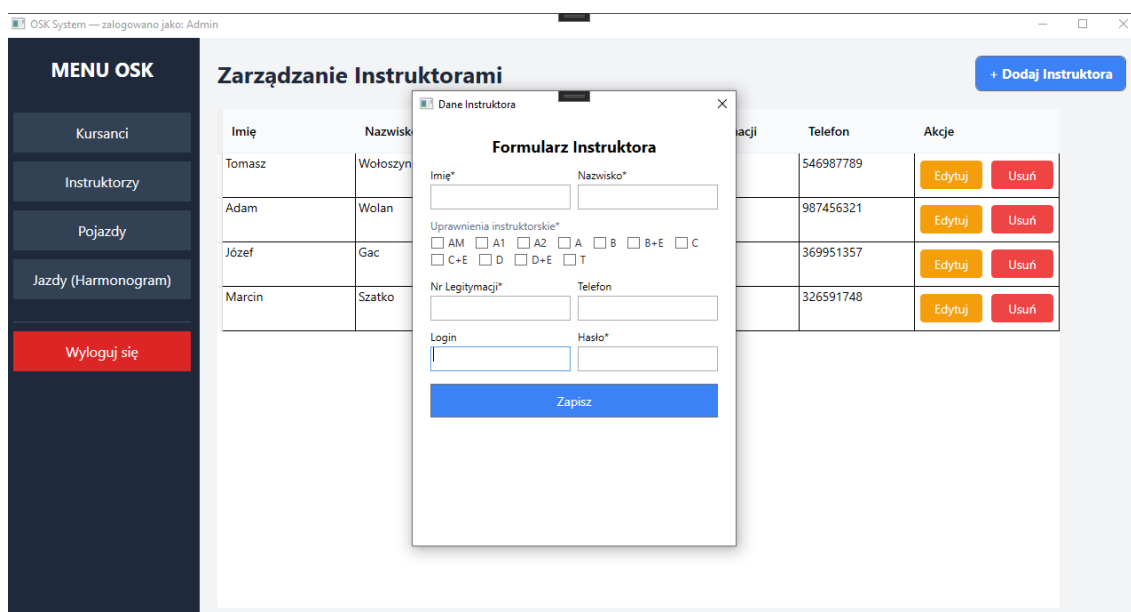
Rysunek 7.5. Komunikat błędu walidacji numeru PESEL

## 7.4. Moduł zarządzania instruktorami

Moduł instruktorów (rys. 7.6) jest analogiczny do modułu kursantów. Formularz dodawania (rys. 7.7) zawiera dodatkowo pole numeru legitymacji, numeru telefonu oraz checkboksy kategorii uprawnień (A, B, C, D i inne). Przy edycji pole hasła pozostaje puste – brak zmiany oznacza zachowanie dotychczasowego hasła.



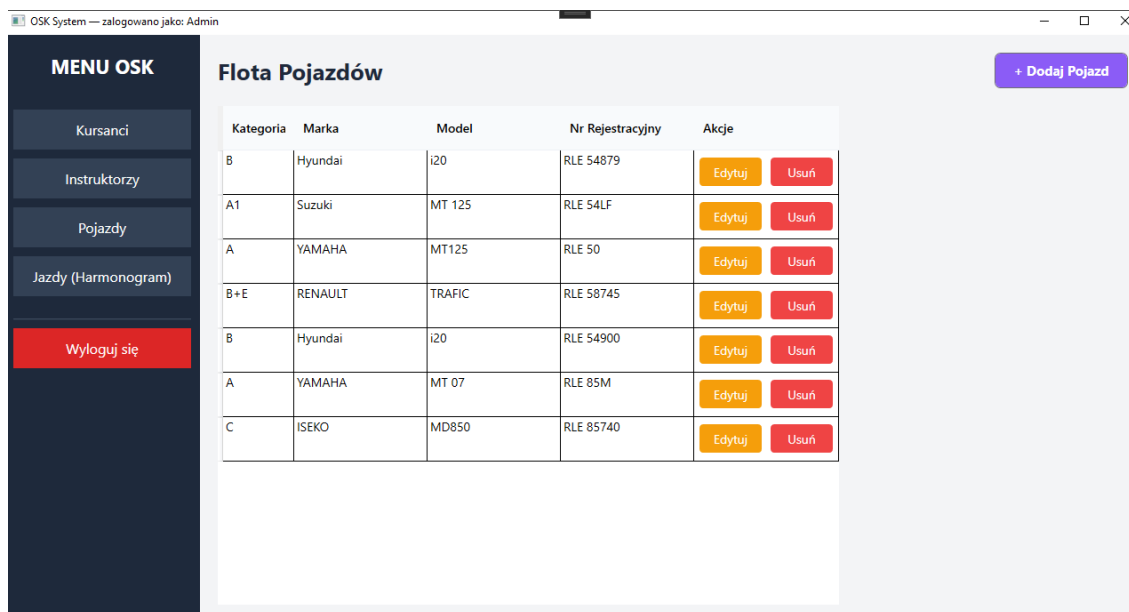
Rysunek 7.6. Lista instruktorów



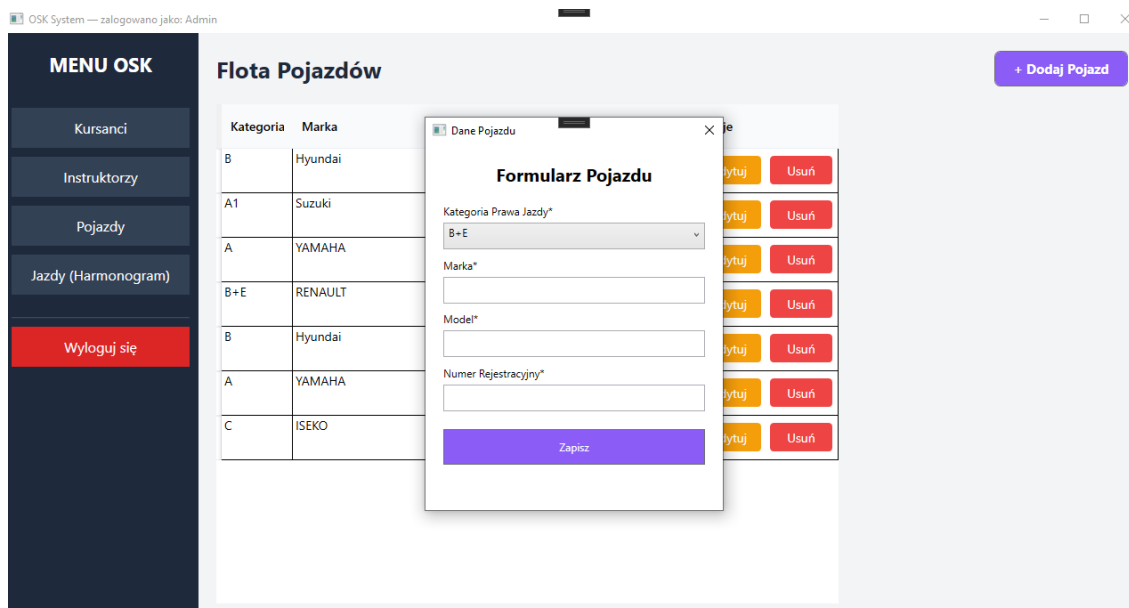
Rysunek 7.7. Formularz dodawania instruktora z checkboksami uprawnień

## 7.5. Moduł zarządzania pojazdami

Ekran pojazdów (rys. 7.8) umożliwia dodawanie, edycję i usuwanie pojazdów szkoleniowych. Formularz (rys. 7.9) zawiera pola: marka, model, numer rejestracyjny oraz kategoria pojazdu.



Rysunek 7.8. Lista pojazdów

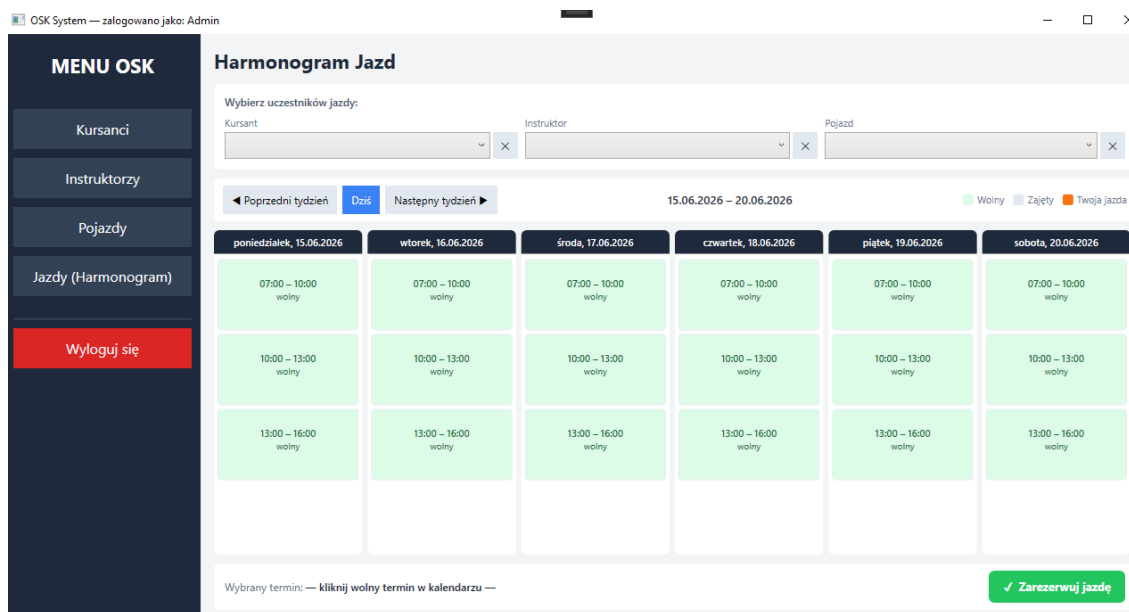


Rysunek 7.9. Formularz dodawania pojazdu

## 7.6. Moduł kalendarza rezerwacji

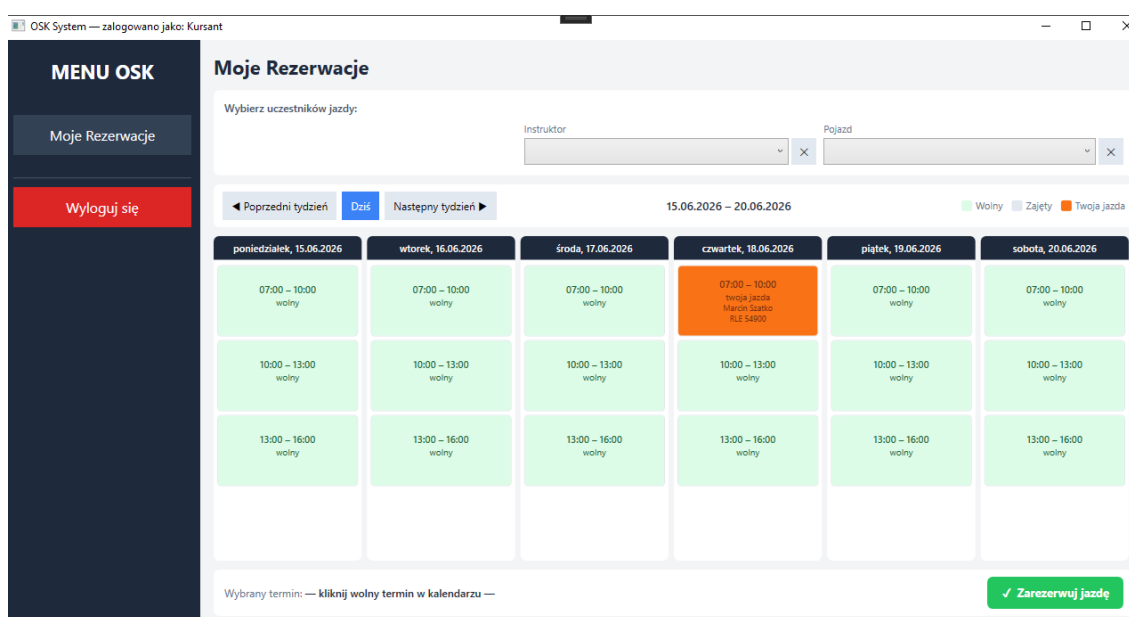
Kalendarz tygodniowy (rys. 7.10) wyświetla siatkę slotów godzinowych w zakresie 8:00–18:00 dla siedmiu dni. Kolory komórek informują o statusie slotu: zielony – własna jazda zalogowanego użytkownika, czerwony – termin zajęty przez inną osobę, szary – termin wolny do zarezerwowania.

Panel filtrów po lewej stronie pozwala administratorowi ograniczyć widok do wybranego kursanta, instruktora lub pojazdu. Przyciski **Wstecz**, **Dziś** i **Dalej** umożliwiają nawigację po kolejnych tygodniach.

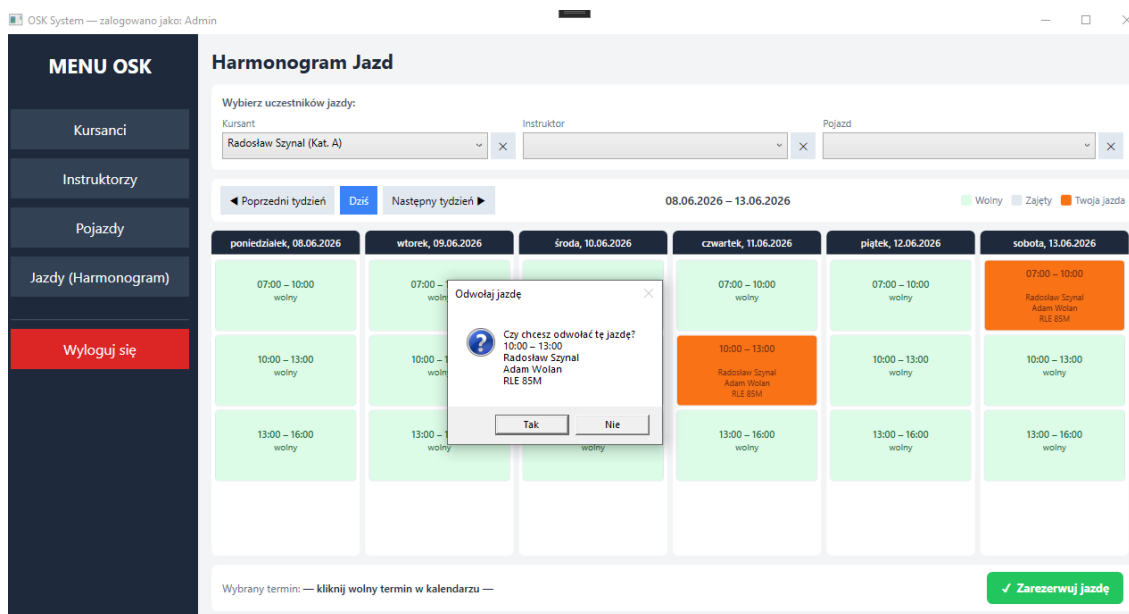


Rysunek 7.10. Kalendarz rezerwacji – widok administratora

Kursant (rys. 7.11) widzi wyłącznie własne zarezerwowane jazdy oraz wolne terminy, które może zarezerwować. Kliknięcie zajętego terminu otwiera okno potwierdzenia odwołania jazdy (rys. 7.12).



Rysunek 7.11. Kalendarz rezerwacji – widok kursanta



Rysunek 7.12. Okno potwierdzenia odwołania rezerwacji

## 7.7. Przykładowy scenariusz działania

Poniżej przedstawiono typowy scenariusz użycia systemu przez trzech różnych użytkowników:

1. Administrator loguje się do systemu, dodaje nowego kursanta wypełniając formularz z numerem PESEL, loginem i hasłem. System automatycznie sprawdza poprawność PESEL i unikalność loginu.
2. Administrator przypisuje do systemu instruktora z uprawnieniami kategorii B i C, a następnie dodaje pojazd do floty szkoleniowej.
3. Kursant loguje się do aplikacji, otwiera kalendarz i wybiera wolny termin w przyszłym tygodniu. System automatycznie dobiera dostępnych instruktorów i pojazdy pasujące do kategorii prawa jazdy kursanta. Kursant potwierdza rezerwację.
4. Instruktor loguje się i sprawdza swój grafik na dany tydzień – widzi zaplanowane terminy jazd.

## 7.8. Scenariusze testowania

Testowanie aplikacji przeprowadzono ręcznie, sprawdzając poprawność działania głównych funkcji systemu.

Tabela 7.1. Scenariusze testowania aplikacji

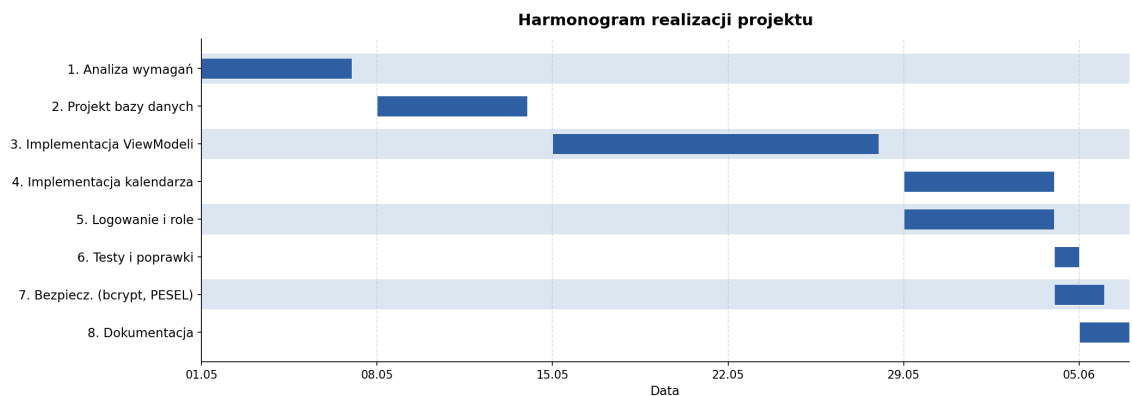
| Scenariusz                                       | Oczekiwany rezultat                      | Wynik |
|--------------------------------------------------|------------------------------------------|-------|
| Logowanie z błędnymi danymi                      | Komunikat „Niepoprawny login lub hasło”  | OK    |
| Dodanie kursanta z błędnym numerem PESEL         | Komunikat o nieprawidłowym numerze PESEL | OK    |
| Dodanie kursanta z już istniejącym loginem       | Komunikat o zajęтым loginie              | OK    |
| Rezerwacja jazdy na termin z przeszłości         | Komunikat o nieprawidłowym terminie      | OK    |
| Próba usunięcia kursanta z zaplanowanymi jazdami | Blokada usunięcia z komunikatem          | OK    |

Wszystkie przetestowane scenariusze działają zgodnie z oczekiwaniami.

## 8. Harmonogram realizacji projektu

Prace nad projektem prowadzone były w trakcie semestru letniego roku akademickiego 2025/2026. Tabela 8.1 przedstawia podział zadań na etapy wraz z przybliżonym czasem realizacji.

Rysunek 8.1 przedstawia te same etapy w formie diagramu Gantta.



Rysunek 8.1. Diagram Gantta harmonogramu realizacji projektu

Tabela 8.1. Harmonogram realizacji projektu

| Etap     | Zakres prac                                                                  | Czas (tygodnie) |
|----------|------------------------------------------------------------------------------|-----------------|
| 1        | Analiza wymagań, dobor technologii, konfiguracja środowiska                  | 1               |
| 2        | Projekt bazy danych, definicja modeli EF Core, pierwsze migracje             | 1               |
| 3        | Implementacja warstwy danych i ViewModeli kursantów, instruktorów i pojazdów | 2               |
| 4        | Implementacja kalendarza rezerwacji (ViewModel + widok XAML)                 | 2               |
| 5        | Implementacja logowania i kontroli dostępu ról                               | 1               |
| 6        | Testy funkcjonalne, naprawa błędów, refaktoryzacja                           | 1               |
| 7        | Bezpieczeństwo: bcrypt, walidacja PESEL, TOCTOU, indeksy unikalne            | 1               |
| 8        | Pisanie dokumentacji, końcowa weryfikacja                                    | 1               |
| łącznie: |                                                                              | 10              |

## 9. Instrukcja uruchomienia projektu

### 9.1. Wymagania systemowe

Do uruchomienia aplikacji niezbędne są:

- System operacyjny: Windows 10 lub Windows 11 (64-bitowy),
- **.NET 10 Runtime** – dostępny na <https://dot.net/download>,
- **SQL Server LocalDB 2019/2022** – instalowany razem z Visual Studio lub oddzielnie jako część paczki *SQL Server Express LocalDB*.

### 9.2. Uruchomienie aplikacji

1. Otworzyć plik `OskSystem.sln` w Visual Studio 2022.
2. Kliknąć **Start** (F5).
3. Przy pierwszym uruchomieniu baza danych zostanie utworzona automatycznie.

### 9.3. Dane logowania

Aplikacja jest wstępnie skonfigurowana z domyślnym kontem administratora:

**Tabela 9.1.** Domyślne dane logowania

| Rola          | Login | Hasło |
|---------------|-------|-------|
| Administrator | admin | admin |

Konta kursantów i instruktorów tworzone są przez administratora z poziomu aplikacji po pierwszym zalogowaniu.

## 10. Podsumowanie i wnioski

### 10.1. Ocena realizacji celu

Wszystkie założone cele projektowe zostały osiągnięte. Aplikacja umożliwia pełne zarządzanie ośrodkiem szkolenia kierowców w zakresie obsługi kursantów, instruktorów, pojazdów oraz rezerwacji jazd. System działa poprawnie dla trzech ról użytkowników.

### 10.2. Wnioski

- Podział aplikacji na warstwy znacząco ułatwia rozbudowę i wprowadzanie zmian – modyfikacja jednej części nie wpływa na pozostałe.
- Użycie gotowych bibliotek do obsługi bazy danych i szyfrowania haseł pozwoliło skupić się na funkcjonalnościach aplikacji zamiast na niskopoziomowych detalach technicznych.

### 10.3. Napotkane problemy

- **Kalendarz rezerwacji** – budowa siatki tygodniowej z dynamicznym kolorowaniem terminów okazała się najtrudniejszą częścią projektu. Wymagała odpowiedniego generowania komponentów UI w kodzie oraz obsługi filtrowania danych w czasie rzeczywistym.
- **Integralność danych** – obsługa próby usunięcia rekordu powiązanego z rezerwacjami wymagała dodatkowej walidacji po stronie aplikacji, ponieważ sam komunikat bazodanowy nie był wystarczająco czytelny dla użytkownika.

### 10.4. Możliwości rozbudowy

W przyszłości aplikacja może zostać rozszerzona o:

- moduł płatności i generowania faktur za kursy,
- raporty i statystyki eksportowane do PDF lub Excel,
- powiadomienia e-mail lub SMS o zaplanowanych jazdach,
- wersję webową dostępną z przeglądarki.

### 10.5. Podsumowanie

Projekt stanowi pełnofunkcjonalną aplikację do zarządzania ośrodkiem szkolenia kierowców, zrealizowaną z użyciem technologii poznanych podczas zajęć. Aplikacja jest gotowa do użycia i stanowi dobrą podstawę do dalszej rozbudowy.

## Bibliografia

- [1] damian6971. Programowanie obiektowe 2 – repozytorium projektu. Dostępne online: [https://github.com/damian6971/projekt\\_ProgramowanieObiektowe2\\_Csharp\\_SystemZarzadzaniaOSK](https://github.com/damian6971/projekt_ProgramowanieObiektowe2_Csharp_SystemZarzadzaniaOSK).
- [2] Microsoft. Entity framework core – dokumentacja. Dostępne online: <https://learn.microsoft.com/pl-pl/ef/core/>. Dostęp: 2026-06-01.
- [3] Microsoft. Windows presentation foundation (WPF) – dokumentacja. Dostępne online: <https://learn.microsoft.com/pl-pl/dotnet/desktop/wpf/>. Dostęp: 2026-06-01.
- [4] Kyle Werther. BCrypt.Net-Next – biblioteka do hashowania haseł dla .net. Dostępne online: <https://github.com/BcryptNet/bcrypt.net>.

# Spis rysunków

|      |                                                                   |    |
|------|-------------------------------------------------------------------|----|
| 3.1  | Diagram klas . . . . .                                            | 12 |
| 5.1  | Diagram ERD bazy danych systemu OSK . . . . .                     | 14 |
| 7.1  | Ekran logowania . . . . .                                         | 18 |
| 7.2  | Główne okno aplikacji – widok administratora . . . . .            | 19 |
| 7.3  | Lista kursantów . . . . .                                         | 19 |
| 7.4  | Formularz dodawania kursanta . . . . .                            | 20 |
| 7.5  | Komunikat błędu walidacji numeru PESEL . . . . .                  | 20 |
| 7.6  | Lista instruktorów . . . . .                                      | 21 |
| 7.7  | Formularz dodawania instruktora z checkboxami uprawnień . . . . . | 21 |
| 7.8  | Lista pojazdów . . . . .                                          | 22 |
| 7.9  | Formularz dodawania pojazdu . . . . .                             | 22 |
| 7.10 | Kalendarz rezerwacji – widok administratora . . . . .             | 23 |
| 7.11 | Kalendarz rezerwacji – widok kursanta . . . . .                   | 23 |
| 7.12 | Okno potwierdzenia odwołania rezerwacji . . . . .                 | 24 |
| 8.1  | Diagram Gantta harmonogramu realizacji projektu . . . . .         | 25 |

# Spis tabel

|     |                                             |    |
|-----|---------------------------------------------|----|
| 2.1 | Wymagania niefunkcjonalne systemu . . . . . | 10 |
| 3.1 | Struktura katalogów projektu . . . . .      | 11 |
| 4.1 | Narzędzia użyte w projekcie . . . . .       | 13 |
| 7.1 | Scenariusze testowania aplikacji . . . . .  | 24 |
| 8.1 | Harmonogram realizacji projektu . . . . .   | 25 |
| 9.1 | Domyślne dane logowania . . . . .           | 26 |

## Spis listingów

|     |                                                               |    |
|-----|---------------------------------------------------------------|----|
| 6.1 | Weryfikacja hasła bcrypt podczas logowania . . . . .          | 15 |
| 6.2 | Algorytm weryfikacji sumy kontrolnej PESEL . . . . .          | 15 |
| 6.3 | Zabezpieczenie przed wyścigiem danych przy usuwaniu . . . . . | 15 |
| 6.4 | Sprawdzanie unikalności loginu przy edycji . . . . .          | 16 |